Course Description (IIIT-Delhi CSE course template)

Code	
Department	CSE
Name	Software Development using AI
Credits	4
OfferedTo	UG
Description	The course introduces AI-native software development where developers collaborate with coding agents. It examines how to define intent, provide effective model context, and structure workflows that integrate AI tools. Students will learn to configure agent-assisted environments, translate unstructured requirements into executable specifications, analyse execution traces to diagnose failures, and assess automated testing pipelines against reliability criteria. Instruction combines lectures with laboratory sessions and a semester-long project, enabling hands-on practice of workflow design, tool integration, and reliable deployment.
Mandatory Prereg	CSE201 (or CSE600A)
Desired Prereq	None
Other Prereq	Strong programming and software development fundamentals; familiarity with advanced programming and basic software engineering concepts
Antireq	None

Course Outcomes

CO #	Outcome	BL #
CO1	Configure AI-assisted coding agents and tool integrations using model context protocols, for a given development environment, such that all public integration tests pass and no configuration warnings are reported.	BL3
CO2	Translate unstructured product requirements into executable specifications, using spec-driven development frameworks, covering every use case in the supplied requirements document.	BL3
CO3	Analyze the execution traces produced by coding agents during iterative loop engineering tasks, identifying at least three distinct failure causes with supporting log evidence.	BL4
CO4	Assess automated testing and evaluation pipelines for AI-native software factories, justifying each design decision against at least one stated reliability requirement.	BL5

2 of 4 COs target BL4–BL6 (template: at least half).

Approval Date	(new course: left empty)
Proposing Dept	CSE

Suggested Topics

List Of Topics	Relevant CO	Assignment/lab/tutorial	Average load (h/week)
Developer workflows and AI-assisted coding	CO1	Quiz 1; Mid-sem exam; End-sem exam; Tutorial (week 1)	8
Model context protocols and architecture	CO1	Quiz 1; Mid-sem exam; End-sem exam; Tutorial (week 2)	9
Coding agents and context engineering	CO1	Quiz 1; Assign 1; Mid-sem exam; End-sem exam; Tutorial (week 3)	10
Custom tool integrations using model context protocols	CO1	Assign 1; Mid-sem exam; End-sem exam; Lab (week 3); Lab (week 4); Lab (week 5)	13
Standards and specification guidelines for use cases	CO2	Assign 2; Mid-sem exam; End-sem exam; Tutorial (week 4)	8
Software requirements and requirements engineering	CO2	Assign 2; Mid-sem exam; End-sem exam	9
Spec-driven development frameworks	CO2	Assign 2; Assign 3; Mid-sem exam; End-sem exam; Tutorial (week 5)	10
Requirements to executable specifications and code generation	CO2	Assign 3; Mid-sem exam; End-sem exam; Lab (week 6); Tutorial (week 6)	13
Iterative development and loop engineering	CO3	LabEval 1; End-sem exam; Lab (week 7); Tutorial (week 7)	11

List Of Topics	Relevant CO	Assignment/lab/tutorial	Average load (h/week)
Iterative development logs and software factory outputs	CO3	LabEval 1; End-sem exam; Lab (week 8); Tutorial (week 8)	12
Software factories and execution trace analysis	CO3	LabEval 1; End-sem exam; Lab (week 9); Tutorial (week 9)	13
Testing and validation in iterative development	CO3	LabEval 1; End-sem exam; Lab (week 10); Tutorial (week 10)	11
Automated testing and evaluation mechanisms	CO4	Project Milestone; End-sem exam; Tutorial (week 11)	9
Pipeline evaluation mechanisms and metrics	CO4	Project Milestone; End-sem exam	9
Reliable and safe trustworthy software architectures	CO4	Project Milestone; End-sem exam; Lab (week 11); Tutorial (week 12)	13
Deployment and reliability metric evaluation	CO4	End-sem exam; Lab (week 12); Lab (week 13); Tutorial (week 13)	12

Average load = expected hours of weekly work outside lectures, including labs, tutorials, homework and self-study (template: 10–12 h for a 4-credit course). Mean over topics: 10.6 h.

Suggested Plan of Labs

List of Lab Topics	Relevant CO	Hardware/software platform
Week 3: Custom tool integration lab: Implement a custom tool integration using model context protocols and verify all tests pass.	CO1	Student development machines, AI coding tools/APIs
Week 4: Hands-on reinforcement lab: Extend the previous integration, adding error handling and logging.	CO1	Student development machines, AI coding tools/APIs
Week 5: Spec-driven practice lab: Create a simple spec-driven module based on earlier requirements and generate skeleton code.	CO1	Student development machines, software development environments
Week 6: Executable specification lab: Translate given module requirements into executable specifications and run code generation.	CO2	Student development machines, software development environments
Week 7: Iterative loop engineering lab: Run an iterative development loop, capture logs, and annotate control-flow steps.	CO3	Student development machines, TA support
Week 8: Defect identification lab: Analyse iterative development logs to locate seeded defects and propose fixes.	CO3	Student development machines, TA support
Week 9: Trace analysis lab: Examine execution traces, identify three failure causes and document evidence.	CO3	Student development machines, TA support
Week 10: Testing pipeline lab: Apply testing and validation techniques to an iterative development artifact.	CO3	Student development machines, TA support
Week 11: Reliability assessment lab: Assess an AI-native testing pipeline against reliability requirements and justify decisions.	CO4	Student development machines, AI coding tools/APIs
Week 12: Deployment metrics lab: Evaluate deployment and reliability metrics for a multi-component project with live walkthroughs.	CO4	Student development machines, AI coding tools/APIs
Week 13: Final reliability lab: Perform a second reliability scenario walkthrough, focusing on metric interpretation.	CO4	Student development machines, AI coding tools/APIs

Suggested Assessment Plan

Type of Evaluation	% Contribution in Grade
Quiz [proctored]	10
Assignment (3)	20
Mid-sem exam [proctored]	20
Lab	10
Project	10
End-sem exam [proctored]	30
Total	**100**

Proctored evaluation: 60% (template: at least 40%).

Suggested Resources

Name/Title	Type
Yujia Li, David Choi, Junyoung Chung, Nate Kushman (2022). Competition-level code generation with AlphaCode. DOI 10.1126/science.abq1158	Reference (paper)
Andrii Tkachuk (2026). SPECIFICATION-DRIVEN DEVELOPMENT AS STRUCTURED APPROACH FOR SOURCE CODE GENERATION. DOI 10.52058/2786-6025-2026-4(58)-2631-2644	Reference (paper)
Anthony Shaw, Amin Beheshti (2026). Complexity Backpressure for AI Coding Agents: A Cross-Model Study on Substantive Software Tasks. DOI 10.1109/sse72781.2026.00047	Reference (paper)
S. Lakshmi, D. Helen, G. Sambasivam (2025). 8 Redefining and Transforming Software Development with Generative AI. DOI 10.1515/9783111677798-008	Reference
Fareedh Basha (2026). Query Tuning Framework Using the Model Context Protocol for Workflows. DOI 10.2139/ssrn.6172059	Reference (paper)
Yeswanth Kumar Polishetty (2026). Autonomous Coding Agents for Software Development Testing and Debugging. DOI 10.2139/ssrn.7515638	Reference (paper)

CO mapping with the programmes offered by the CSE department

Cell = mapping strength (1 low, 2 medium, 3 high); explanations follow each programme table.

B.Tech. (common)	Program Outcome	CO1	CO2	CO3	CO4
PO1	Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.	3		2	
PO2	Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.	3	3	3	3
PO3	Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.		3		2
PO4	Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.			3	
PO5	Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.	1			
PO6	The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.				
PO7	Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.				
PO8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.				
PO9	Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.				
PO10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.		3		3
PO11	Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.				
PO12	Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.				

Explanations: CO1 → PO1: Basic engineering principles are applied when configuring development environments and integration tests. · CO3 → PO1: Applies software engineering fundamentals during trace inspection. · CO1 → PO2: Addressing integration test failures requires minor analytical reasoning. · CO2 → PO2: Requires analyzing unstructured requirements to formulate technical specs. · CO3 → PO2: Reaching substantiated conclusions on agent failures requires deep problem analysis. · CO4 → PO2: Justifying pipeline decisions requires analyzing reliability requirements. · CO2 → PO3: Translating unstructured requirements into executable specifications constitutes designing software system specifications. · CO4 → PO3: Assessing and designing automated testing and evaluation pipelines directly addresses system-level design solutions. · CO3 → PO4: Analyzing execution traces and log evidence to identify failure causes constitutes rigorous data analysis and investigation. · CO1 → PO5: Configuring AI-assisted coding agents and tool integrations directly exercises modern engineering and IT tool usage. · CO2 → PO10: Producing executable specifications entails writing comprehensive design documentation. · CO4 → PO10: Documenting design justifications requires effective professional communication.

B.Tech.(CSE)	Program Outcome	CO1	CO2	CO3	CO4
PSO1	Understanding of theoretical foundations and limits of computing.	3	2	2	3
PSO2	Understanding of computing at different levels of abstraction including circuits and computer architecture, operating systems, algorithms, and applications.				
PSO3	Ability to adapt established models, techniques, algorithms, data structures, etc. for efficiently solving new problems.		2	3	
PSO4	Ability to design, implement, and evaluate computer based system or application to meet the desired needs using modern tools and methodologies.	3	3	3	3
PSO5	Understanding and ability to use advanced techniques and tools in different areas of computing.	1			2

Explanations: CO1 → PSO1: Touches upon protocol foundations. · CO2 → PSO1: Formal specifications touch upon theoretical limits of systems. · CO3 → PSO1: Execution traces reflect computing runtime limitations. · CO4 → PSO1: Reliability requirements touch upon theoretical computing limits. · CO2 → PSO3: Adapts development frameworks to new product requirements. · CO3 → PSO3: Uses trace analysis techniques to solve debugging problems. · CO1 → PSO4: Configuration of coding agents implements tool-based application support. · CO2 → PSO4: Spec-driven development frameworks are modern methodologies for designing applications. · CO3 → PSO4: Evaluates computer-based system performance via execution traces. · CO4 → PSO4: Evaluating testing pipelines for AI-native software factories targets application evaluation needs. · CO1 → PSO5: Uses advanced model context protocols and coding agent integrations. · CO4 → PSO5: Applies advanced techniques for AI software factories.

B.Tech.(CSAI)	Program Outcome	CO1	CO2	CO3	CO4
PSO1	Understanding of foundational topics in Computer Science, Artificial Intelligence, and Machine Learning.	3	2	2	3
PSO2	Understanding of theoretical foundations and limits of Artificial Intelligence and Machine Learning.				
PSO3	Ability to design and implement algorithms and data structures for efficiently solving new problems.	1	3		
PSO4	Ability to model and analyze a variety of problems using appropriate mathematical/computational and AI concepts.		3	3	3
PSO5	Ability of apply and develop AI algorithms to transform large amount of data into intelligent decisions and/or behavior.	1		2	2
PSO6	An understanding of the impact of AI based solutions in an economic, societal, and environment context.				

Explanations: CO1 → PSO1: Applies AI-related foundational concepts. · CO2 → PSO1: Applies foundational computer science modeling concepts. · CO3 → PSO1: Touches upon AI agent foundations. · CO4 → PSO1: Touches upon AI systems foundations. · CO1 → PSO3: Implementing agent tool integrations aligns with algorithmic problem solving. · CO2 → PSO3: Designing executable specifications involves structured problem solving. · CO2 → PSO4: Models product requirements computationally. · CO3 → PSO4: Analyzing AI agent traces is a direct application of computational and AI problem analysis. · CO4 → PSO4: Assesses AI-native system pipelines. · CO1 → PSO5: Configuring AI coding agents applies AI behaviors. · CO3 → PSO5: Analyzes iterative AI agent behaviors. · CO4 → PSO5: Applies AI evaluation concepts to software factories.

MTech(CSE)	Program Outcome	CO1	CO2	CO3	CO4
PO1	An ability to independently carry out research /investigation and development work to solve practical problems	2	3	2	2
PO2	An ability to write and present a substantial technical report/document	2	3	2	3
PO3	Students should be able to demonstrate a degree of mastery over the area as per the specialization of the program. The mastery should be at a level higher than the requirements in the appropriate bachelor program	1	1	2	2
PSO1	An understanding of the theoretical foundations and the limits of computing.	3		2	
PSO2	An ability to adapt existing models, techniques, algorithms, data structures, etc. for efficiently solving problems.		2	3	
PSO3	An ability to design, develop and evaluate new computer based systems for novel applications which meet the desired needs of industry and society.	3	3		3
PSO4	Understanding and ability to use advanced computing techniques and tools.	1			2
PSO5	An ability to undertake original research at the cutting edge of computer science & its related areas.			2	
PSO6	An ability to function effectively individually or as a part of a team to accomplish a state goal.				
PSO7	An understanding of professional and ethical responsibility.				
PSO8	An ability to communicate effectively with a wide range of audience.		3		3
PSO9	An ability to learn independently and engage in lifelong learning.				
PSO10	An understanding of the impact of IT related solutions in an economic, societal, and environment context.				

Explanations: CO1 → PO1: Development work involves configuring coding environments. · CO2 → PO1: Carries out independent specification development. · CO3 → PO1: Investigates practical engineering failures independently using logs. · CO4 → PO1: Carries out investigation into testing pipelines. · CO1 → PO2: Reporting test outcomes satisfies documentation aspects. · CO2 → PO2: Writing complete executable specifications serves as technical documentation. · CO3 → PO2: Log evidence contributes to technical reporting. · CO4 → PO2: Writing design justifications aligns with technical reporting. · CO1 → PO3: Demonstrates baseline specialization mastery. · CO2 → PO3: Demonstrates methodology mastery. · CO3 → PO3: Demonstrates advanced diagnostic mastery. · CO4 → PO3: Demonstrates advanced specialization mastery in AI systems. · CO1 → PSO1: Touches upon protocol limits. · CO3 → PSO1: Traces expose computing limits. · CO2 → PSO2: Adapts requirements engineering models. · CO3 → PSO2: Adapts analysis techniques to software traces. · CO1 → PSO3: Develops environment configurations. · CO2 → PSO3: Designs and develops executable system specifications from requirements. · CO4 → PSO3: Evaluates testing pipelines for novel AI-native software factories. · CO1 → PSO4: Directly employs advanced AI tooling techniques. · CO4 → PSO4: Uses advanced testing and evaluation tooling. · CO3 → PSO5: Investigates cutting-edge AI coding agent traces. · CO2 → PSO8: Communicates requirements via clear executable documentation. · CO4 → PSO8: Communicates design rationales clearly.

Annex: week-by-week schedule

Week	Lecture topics	COs	Assessments
1	• Developer workflows and AI-assisted coding • Model context protocols and architecture	CO1	
2	• Model context protocols and architecture • Coding agents and context engineering	CO1	

Week	Lecture topics	COs	Assessments
3	• Coding agents and context engineering • Custom tool integrations using model context protocols	CO1	Quiz 1
4	• Custom tool integrations using model context protocols • Standards and specification guidelines for use cases • Software requirements and requirements engineering	CO1, CO2	Assign 1 released
5	• Software requirements and requirements engineering • Spec-driven development frameworks	CO2	
6	• Spec-driven development frameworks • Requirements to executable specifications and code generation	CO2	Assign 1 due Assign 2 released
7	• Requirements to executable specifications and code generation • Iterative development and loop engineering	CO2, CO3	Assign 3 released Mid-sem exam
8	• Iterative development and loop engineering • Iterative development logs and software factory outputs	CO3	Assign 2 due Project Milestone released
9	• Software factories and execution trace analysis • Testing and validation in iterative development	CO3	Assign 3 due
10	• Testing and validation in iterative development • Automated testing and evaluation mechanisms	CO3, CO4	LabEval 1 released
11	• Automated testing and evaluation mechanisms • Pipeline evaluation mechanisms and metrics • Reliable and safe trustworthy software architectures	CO4	
12	• Reliable and safe trustworthy software architectures • Deployment and reliability metric evaluation	CO4	LabEval 1 due Project Milestone due
13	• Deployment and reliability metric evaluation	CO4	End-sem exam

Draft generated by ProfsAgent from design run cs601-02. Not an approved IIIT-Delhi course record.